RTK QUERY – COMPLETE NOTES

1. What is RTK Query ?

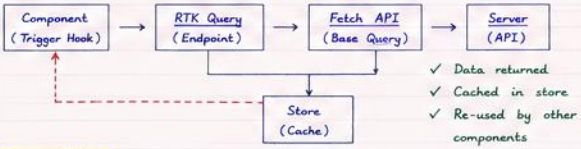
- RTK Query is the data fetching and caching tool built inside Redux Toolkit.
- It simplifies API calls, caching, loading states, error handling, polling, pagination etc.
- Eliminates the need for extra libraries like axios + react-query in most cases.
- Built on top of Redux Toolkit + Immer.

Less code
Auto caching
Loading & error handling built-in
Great DX 👍

2. Core Concepts

- Endpoint → A description of how to fetch (or mutate) data.
- Query → For fetching data (GET).
- Mutation → For changing data (POST, PUT, DELETE).
- Cache → Stores the result of queries in Redux store.
- Tag → Used for cache invalidation & refetching.
- Re-usable Hooks → Auto generated hooks for each endpoint.

3. How RTK Query Works (High Level Flow)



4. Basic Setup

Step 1: Install (if needed)

```
npm i @reduxjs/toolkit react-redux
```

Step 2: Create an API Slice (api/postsApi.js)

```
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react'

export const postsApi = createApi({
  reducerPath: 'postsApi',
  baseQuery: fetchBaseQuery({ baseUrl: 'https://jsonplaceholder.typicode.com' }),
  endpoints: (builder) => ({
    getPosts: builder.query({
      query: () => '/posts',
    }),
  }),
})
```

createApi creates a slice with reducer + middleware automatically

Step 3: Add to Store (store.js)

```
import { configureStore } from '@reduxjs/toolkit'
import { postsApi } from './api/postsApi'

export const store = configureStore({
  reducer: {
    [postsApi.reducerPath]: postsApi.reducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(postsApi.middleware),
})
```

Don't forget to add middleware!

Step 4: Provide Store

```
<Provider store={store}>
  <App />
</Provider>
```

Now RTK Query is ready to use in the app 🎉

5. Endpoints – Query vs Mutation

Query (GET)	Mutation (POST/PUT/DELETE)
• Read / fetch data	• Change / update data
• Cached automatically	• Does not cache result
• Auto refetch on certain events	• You usually invalidate tags
• Uses builder.query()	• Uses builder.mutation()

6. Multiple Endpoints Example

```
export const postsApi = createApi({
  reducerPath: 'postsApi',
  baseQuery: fetchBaseQuery({ baseUrl: 'https://jsonplaceholder.typicode.com' }),
  tagTypes: ['Post'],
  endpoints: (builder) => ({
    getPosts: builder.query({
      query: () => '/posts',
      providesTags: (result) =>
        result
        ? [
            ....result.map(({ id }) => ({ type: 'Post', id })),
            { type: 'Post', id: 'LIST' },
          ]
        : [{ type: 'Post', id: 'LIST' }],
    }),
    addPost: builder.mutation({
      query: (newPost) => ({
        url: '/posts',
        method: 'POST',
        body: newPost,
      }),
      invalidatesTags: [{ type: 'Post', id: 'LIST' }],
    }),
  }),
})
```

providesTags → Tells what data this query provides.
invalidatesTags → Marks cache as stale & triggers refetch.

7. Auto Generated Hooks

Endpoint Type	Example Endpoint	Auto Generated Hook
Query	getPosts	useGetPostsQuery()
Mutation	addPost	useAddPostMutation()

- Hooks are generated based on endpoint name.
- Use them directly in components.

8. Using in Component

Example – Fetching Data

```
import { useGetPostsQuery } from './api/postsApi'

function Posts() {
  const { data, isLoading, isError, error } = useGetPostsQuery()
  if (isLoading) return <p>Loading...</p>
  if (isError) return <p>Error : {error.toString()}</p>

  return (
    <ul>
    {data?.map(post) => (
      <li key={post.id}>{post.title}</li>
    ))}
  )
}
```

data → result
isLoading → loading state
isError → error state
error → error object

9. Using Mutation in Component

```
import { useAddPostMutation } from './api/postsApi'

function AddPost() {
  const [addPost, { isLoading, isSuccess, isError}] = useAddPostMutation()

  const handleAdd = () => {
    addPost({
      title: 'foo',
      body: 'bar',
      userId: 1,
    })
  }

  return (
    <div>
      <button onClick={handleAdd} disabled={isLoading}
        {isLoading ? 'Adding...' : 'Add Post'}
      </button>
      {isSuccess && <p> ✅ Post Added!</p>}
      {isError && <p> ❌ Failed to add post</p>}
    </div>
  )
```

Mutation returns a trigger function + status flags.

10. Cache & Invalidation

- Queries are cached based on endpoint + parameters.
- By default cache is kept as long as component is mounted.
- Use invalidatesTags in mutation to refetch related queries.
- You can also use refetch() manually.

11. Useful Options in Endpoints

Option	Purpose
query	Defines the API endpoint.
providesTags	Defines what data this query provides.
invalidatesTags	Invalidates related cache.
transformResponse	Modifies the response data.
keepUnusedDataFor	Time (in sec) to keep data in cache after component unmount.

12. Benefits

- ✓ Less boilerplate code
- ✓ Built-in caching
- ✓ Auto refetching
- ✓ Loading & error handling
- ✓ Pagination, polling, refetch on focus
- ✓ Tag based cache invalidation
- ✓ Great Developer Experience.

13. Quick Revision

1. createApi() → Create API slice.
2. Define endpoints with builder.query / builder.mutation.
3. Add reducer + middleware to store.
4. Use auto generated hooks in components.
5. Enjoy caching, refetching & less boilerplate! 🎉

Flow

Component
↓
Hook (useXXX)
↓
Endpoint
↓
Base Query
↓
Server
↓
Store (Cache)

★ RTK Query = Simplicity + Power + Performance ★